

Virtual Warehouses :

Virtual Warehouse in Snowflake is **compute power**, not storage. Think of it like a **temporary engine** that runs queries, that used to process queries without moving data.

- Run SQL queries
- Load data (COPY, INSERT)
- Transform data
- Execute tasks, procedures, streams
- Data is not stored here — data stays in Snowflake storage.

Warehouse Sizes:

Size	When to Use
X-Small	Small queries, testing
Small	Daily reporting
Medium	BI dashboards
Large+	Heavy transformations

Bigger size = more power = faster queries = higher cost

Multi-Cluster Warehouses :

Used when **many users run queries at the same time**.

- Prevents queueing
- Improves concurrency
- Cost increases only when clusters are active
- MIN_CLUSTER = 1 ; MAX_CLUSTER = 3

```
86 CREATE WAREHOUSE sales_wh
87 WITH WAREHOUSE_SIZE='MEDIUM'
88 MIN_CLUSTER_COUNT=1
89 MAX_CLUSTER_COUNT=3;
90
```

Results (just now)

Table	Chart	1 row	150ms	⌵	⌵
status					
1		Warehouse SALES_WH successfully created.			

Considerations :

- Things to think about before creating/using a warehouse. If you keep a warehouse running without queries → money is wasted . So enable AUTO_SUSPEND.

Main usage:

- Cost
- Performance
- Auto-suspend
- Size

Working with Virtual Warehouses :

- Create a Warehouse
- Use a Warehouse
- Start (Resume) Warehouse
- Stop (Suspend) Warehouse
- Resize Warehouse
- Enable Multi-Cluster
- Monitor Warehouse
- Drop Warehouse

Create a Warehouse : Purpose to Create compute to run queries

```
92 CREATE OR REPLACE WAREHOUSE my_wh
93 WITH
94 WAREHOUSE_SIZE = 'SMALL'
95 AUTO_SUSPEND = 60
96 AUTO_RESUME = TRUE;
97
```

results (just now)	
Table	Chart
1 row 143ms	
status	
1	Warehouse MY_WH successfully created.

Use a Warehouse : Tell Snowflake which warehouse to use

```
--
98 USE WAREHOUSE my_wh;
99
```

results (just now)	
Table	Chart
1 row 36ms	
status	
1	Statement executed successfully.

Start (Resume) Warehouse : Manually start if needed

```
100 | ALTER WAREHOUSE my_wh RESUME;
```

```
101
```

Results (just now)	
Table	Chart
1 row	64ms
id	status
1	Statement executed successfully.

Stop (Suspend) Warehouse : Stop compute to save cost

```
102 | ALTER WAREHOUSE my_wh SUSPEND;
```

```
103
```

Results (just now)	
Table	Chart
1 row	225ms
id	status
1	Statement executed successfully.

Resize Warehouse : Increase or decrease power

```
104 | ALTER WAREHOUSE my_wh SET WAREHOUSE_SIZE = 'LARGE';
```

```
105
```

Results (just now)	
Table	Chart
1 row	62ms
id	status
1	Statement executed successfully.

Enable Multi-Cluster : Handle many users at once

```
107 | ALTER WAREHOUSE my_wh
```

```
108 | SET MIN_CLUSTER_COUNT = 1
```

```
109 | MAX_CLUSTER_COUNT = 3;
```

```
110
```

Results (just now)	
Table	Chart
1 row	62ms
id	status
1	Statement executed successfully.

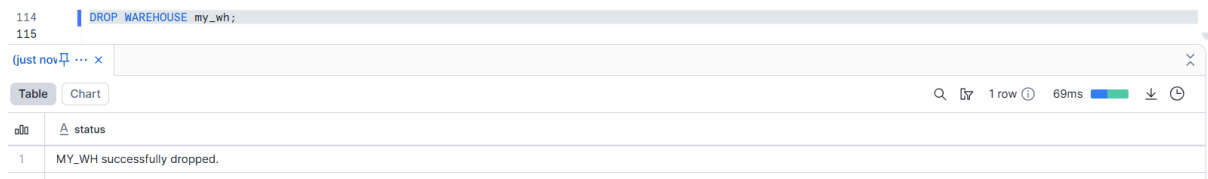
Monitor Warehouse : Check usage & performance

```
112 | SHOW WAREHOUSES;
```

```
113
```

Results (just now)											
Table	Chart	8 rows	132ms								
id	name	state	type	size	# min_cluster_count	# max_cluster_count	# started_clusters	# running	# queued	is_default	is_current
1	COMPUTE_WH	STARTED	STANDARD	X-Small	1	1	1	0	0	Y	N
2	DEMO_WH	SUSPENDED	STANDARD	Large	1	1	0	0	0	N	N
3	MY_WAREHOUSE	SUSPENDED	STANDARD	X-Small	1	1	0	0	0	N	N
4	MY_WH	SUSPENDED	STANDARD	Large	1	3	0	0	0	N	Y
5	SALES_WH	SUSPENDED	STANDARD	Medium	1	3	0	0	0	N	N
6	SMALL_WH	SUSPENDED	STANDARD	Small	1	3	0	0	0	N	N
7	SNOWFLAKE_LEARN	SUSPENDED	STANDARD	X-Small	1	1	0	0	0	N	N

Drop Warehouse : Remove when not needed



The screenshot shows a Snowflake SQL interface. At the top, the query is `DROP WAREHOUSE my_wh;`. Below the query, there are tabs for 'Table' and 'Chart'. The 'Table' tab is selected, showing a single row with the status 'MY_WH successfully dropped.' The table has a header row with a 'status' column. The interface also shows a search bar, a filter icon, and a status bar indicating '1 row' and '69ms'.

	status
1	MY_WH successfully dropped.

Simple Flow : Create → Use → Run → Resize → Monitor → Suspend → Drop

Next-Generation Standard Warehouses :

- **Next-Generation Standard Warehouses** are the **new and improved default compute warehouses** in Snowflake.
- They are **faster, smarter, and more efficient** than older standard warehouses — and you don't need to change your SQL.

For Example : A company has a **sales table with 500 million rows**. Business users run daily reports.

Before (Old Standard Warehouse)

- Warehouse size: **Medium**
- Query runtime: **30 seconds**
- Sometimes runs out of memory
- Need to resize to **Large** (more cost)

After (Next-Generation Standard Warehouse)

- Warehouse size: **Same (Medium)**
- Query runtime: **18 seconds**
- Better memory handling
- No resize needed
- Lower cost

Interactive Tables and Warehouses :

Interactive Tables

- Tables that users **query and analyze in real time**.
- Used for **dashboards, reports, ad-hoc analysis**, and user-facing apps.
- Designed for **fast SELECT queries** with frequent access.

Example : A **sales_dashboard** table queried by business users every few seconds to see today's revenue.

Interactive Warehouses

- Compute warehouses optimized for **low-latency, quick response** queries.
- Usually **small to medium size**, kept **running during business hours**.
- Handle **many concurrent users** efficiently.

Example: A **WH_REPORTING** warehouse used by Power BI/Tableau for live dashboards.

How They Work Together

- **Interactive tables** store the data.
- **Interactive warehouses** provide the compute power to query that data instantly.
- Separation of compute and storage ensures **no impact on ETL/batch jobs**.

View Concepts:

Views : A virtual table created using a SQL query. A **view** is used to **simplify, secure, and standardize access to data** without copying it.

These are key points:

- Stores only the query, not data
- Always shows latest data
- Uses data from base tables

For Example:

- Create a Base Table
- Insert Sample Data
- Create a View (View shows only high-value orders)
- Query the View

```
1 CREATE OR REPLACE TABLE orders (  
2   order_id INT,  
3   customer_name STRING,  
4   amount NUMBER,  
5   order_date DATE  
6 );  
7 INSERT INTO orders VALUES  
8 (1, 'Ravi', 5000, '2024-12-01'),  
9 (2, 'Anu', 3000, '2024-12-05'),  
10 (3, 'John', 8000, '2024-12-10');  
11  
12 CREATE OR REPLACE VIEW high_value_orders AS  
13 SELECT order_id, customer_name, amount  
14 FROM orders  
15 WHERE amount > 4000;  
16  
17 SELECT * FROM high_value_orders;  
18
```

Results (just now)

Table				Chart			
	#	ORDER_ID		#	CUSTOMER_NAME		#
							AMOUNT
1		1	Ravi				5000
2		3	John				8000

Types of Views :

View Type	Stores Data	Security	Performance
Standard View	✗ No	Medium	Normal
Secure View	✗ No	🔒 High	Normal
Materialized View	✓ Yes	Medium	🚀 Fast
Recursive View	✗ No	Medium	Normal
Semantic View	✗ No	Medium	Depends

Standard View : A virtual table created using a SQL query (Is like normal view).

Secure View :

- Hides the view's SQL logic from users.
- Use case : Share sensitive data safely.

```
19 CREATE SECURE VIEW secure_salary_view AS
20 SELECT order_id, amount
21 FROM orders;
```

Results (1 minute ago)

Table Chart 1 row 166ms

status
View SECURE_SALARY_VIEW successfully created.

Materialized View :

- Stores pre-computed results physically.
- Use case : Fast reporting and dashboards.

```
3 CREATE MATERIALIZED VIEW daily_sales_mv AS
4 SELECT order_date, SUM(amount) AS total_sales
5 FROM orders
6 GROUP BY order_date;
```

Results (just now)

Table Chart 1 row 821ms

status
Materialized view DAILY_SALES_MV successfully created.

Recursive View :

- A **recursive view** is used to handle **hierarchical data** where rows are related to each other.
- **Hierarchical** means **data arranged in levels**, where each item has a **parent-child relationship**.

```
18 CREATE OR REPLACE TABLE employees (  
19     emp_id INT,  
20     manager_id INT,  
21     name STRING  
22 );  
23  
24 INSERT INTO employees VALUES  
25 (1, NULL, 'CEO'),  
26 (2, 1, 'Manager A'),  
27 (3, 1, 'Manager B'),  
28 (4, 2, 'Developer A'),  
29 (5, 2, 'Developer B'),  
30 (6, 3, 'Tester');
```

sults (just now)

Table	Chart	1 row	211ms	⬇	🕒
1	number of rows inserted	6			

```
43 CREATE OR REPLACE VIEW org_hierarchy AS  
44 WITH RECURSIVE emp_tree AS (  
45     -- Top Level (CEO)  
46     SELECT emp_id, manager_id, name  
47     FROM employees  
48     WHERE manager_id IS NULL  
49  
50     UNION ALL  
51  
52     -- Subordinates  
53     SELECT e.emp_id, e.manager_id, e.name  
54     FROM employees e  
55     JOIN emp_tree t  
56     ON e.manager_id = t.emp_id  
57 )  
58 SELECT * FROM emp_tree;  
59  
60 SELECT * FROM org_hierarchy;  
61
```

Results (just now)

Table	Chart	6 rows	213ms	⬇	🕒
EMP_ID	MANAGER_ID	NAME			
1	1	CEO			
2	2	Manager A			
3	3	Manager B			
4	4	Developer A			

Semantic View :

- A semantic view provides a business-friendly abstraction layer that defines consistent metrics and dimensions for analytics.
- Steps : Base Table, Semantic View Query, How Business Users Query It.

```
62 CREATE OR REPLACE TABLE orders (  
63   order_id INT,  
64   customer STRING,  
65   region STRING,  
66   amount NUMBER,  
67   order_date DATE  
68 );  
69  
69 INSERT INTO orders VALUES  
70 (1, 'Ravi', 'South', 5000, '2024-12-01'),  
71 (2, 'Anu', 'North', 3000, '2024-12-02'),  
72 (3, 'John', 'South', 7000, '2024-12-03');  
73
```

Results (just now)

Table Chart 1 row 211ms

#	number of rows inserted
1	3

```
74 CREATE OR REPLACE VIEW sales_semantic_view AS  
75 SELECT  
76   region AS region, -- Dimension  
77   order_date AS order_date, -- Dimension  
78   COUNT(order_id) AS order_count, -- Measure  
79   SUM(amount) AS total_revenue -- Measure  
80 FROM orders  
81 GROUP BY region, order_date;  
82  
83 SELECT region, total_revenue  
84 FROM sales_semantic_view;  
85
```

Results (just now)

Table Chart 3 rows 267ms

#	REGION	TOTAL_REVENUE
1	South	5000
2	South	7000
3	North	3000

Raw data → Semantic View → Business-friendly data

